Documentación técnica — ClienTrack (Mini CRM)

Materia: Desarrollo Web, Universidad EAN
Autor(a): Yiseth Karina Gomez Hernandez
Fecha: Noviembre de 2026

1. Motivo

Muchos emprendedores, vendedores independientes y pequeños negocios atienden a sus clientes por WhatsApp, llamadas o correo, y llevan los pendientes “de memoria” o en papel. Así se olvidan solicitudes, se pasan fechas de entrega y no se sabe qué cliente está esperando respuesta.

Las herramientas profesionales de CRM (*Customer Relationship Management*), como Salesforce o HubSpot, resuelven este problema, pero son costosas y complejas para un negocio pequeño. **ClienTrack** es una versión mínima y gratuita de esa idea: un CRM de bolsillo que funciona en el celular, sin internet.

2. Objetivo

Desarrollar una aplicación Android que permita a un usuario:

1. Registrarse e iniciar sesión de forma segura.
2. Administrar su lista de clientes.
3. Crear, consultar, editar, cerrar y eliminar casos asociados a cada cliente, con prioridad, estado y fecha límite.
4. Encontrar rápidamente la información con búsqueda y filtros, y ver un resumen de su carga de trabajo.

3. Alcance

Incluye

- Pantalla de bienvenida con instrucciones de uso.
- Registro e inicio de sesión con validaciones y sesión persistente.

- CRUD completo de **clientes** (crear, leer, actualizar, eliminar).
- CRUD completo de **casos**, más la acción rápida “Marcar como cerrado”.
- Búsqueda en tiempo real por asunto, cliente o empresa, y filtro por estado.
- Resumen con contadores: abiertos, en proceso, cerrados y vencidos.
- Detección automática de casos vencidos.
- Almacenamiento local en SQLite.

No incluye (posibles mejoras futuras)

- Sincronización en la nube ni acceso desde varios dispositivos (por ejemplo, con Firebase).
- Notificaciones o recordatorios de fechas límite.
- Recuperación de contraseña por correo.
- Varios usuarios trabajando sobre los mismos clientes.

4. Tecnologías utilizadas

Tecnología	Uso en el proyecto
Android Studio	Entorno de desarrollo
Java 17	Lenguaje de programación de la lógica
XML	Diseño de las pantallas (layouts), colores, estilos y textos
SQLite (`SQLiteOpenHelper`)	Base de datos local relacional
Material Components (Material 3)	Componentes visuales: botones, campos de texto, chips, grupos segmentados, calendario, hoja inferior, diálogos
Inter (licencia OFL)	Tipografía de toda la app, incluida en `res/font`
RecyclerView	Listas de casos y clientes
SharedPreferences	Guardar la sesión del usuario
SHA-256 (`MessageDigest`)	Cifrado de contraseñas
JUnit 4 y **Espresso**	Pruebas unitarias y pruebas de interfaz

Las versiones exactas están en el `README.md`.

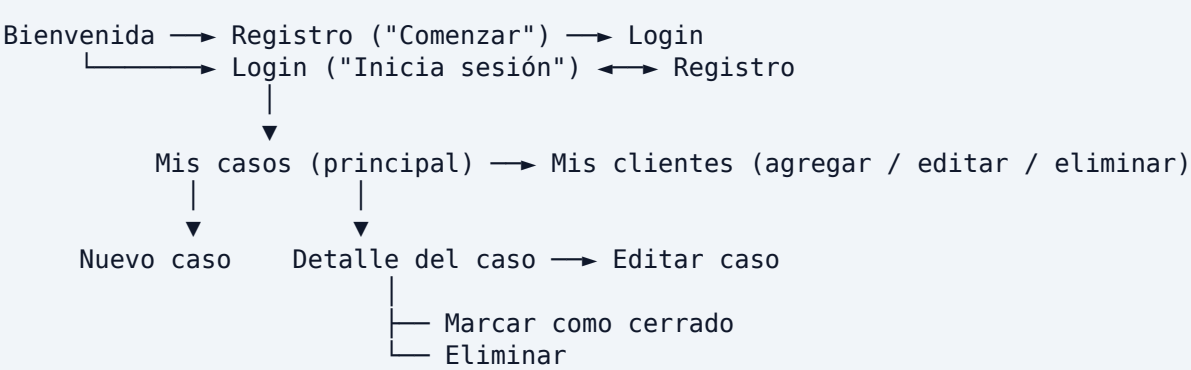
5. Arquitectura

El código está organizado por responsabilidades:

```
co.edu.ean.clientrack
├── ui/        Pantallas (Activities): acciones del usuario y vista
├── adapter/   Convierten listas de datos en filas en pantalla (ListAdapter)
├── model/     Clases de datos: Usuario, Cliente, Caso
├── data/      DatabaseHelper: tablas y consultas SQL
└── util/      Sesión, validaciones/cifrado, colores, mensajes y formatos de fecha
```

Las pantallas nunca escriben SQL directamente. Siempre llaman a métodos de `DatabaseHelper`, por ejemplo `db.buscarCasos(...)` o `db.insertarCliente(...)`. Así la lógica de datos queda en un solo lugar y es fácil de mantener.

Flujo de navegación



6. Base de datos

La base de datos se llama `clientrack.db` y tiene **tres tablas relacionadas**.

6.1 Modelo entidad-relación



- **Un usuario tiene muchos clientes** (`clientes.usuario_id → usuarios.id`).
- **Un cliente tiene muchos casos** (`casos.cliente_id → clientes.id`).
- **Un usuario tiene muchos casos** (`casos.usuario_id → usuarios.id`). Esta llave se repite en `casos` para que las consultas filtren directamente por usuario y sea imposible ver casos de otra persona.

6.2 Tabla `usuarios`

Guarda las cuentas que se crean en la pantalla de registro.

Columna	Tipo	Restricciones	Descripción
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Identificador
nombre	TEXT	NOT NULL	Nombre completo
correo	TEXT	NOT NULL, **UNIQUE**	Correo de acceso (no se puede repetir)
password_hash	TEXT	NOT NULL	Contraseña cifrada con SHA-256
fecha_registro	TEXT	NOT NULL	Fecha de creación (yyyy-MM-dd)

6.3 Tabla `clientes`

Columna	Tipo	Restricciones	Descripción
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Identificador
usuario_id	INTEGER	NOT NULL, FK → usuarios(id), ON DELETE CASCADE	Dueño del cliente
nombre	TEXT	NOT NULL	Nombre del cliente
empresa	TEXT	—	Empresa (opcional)
correo	TEXT	—	Correo (opcional)
telefono	TEXT	—	Teléfono (opcional)

6.4 Tabla `casos`

Columna	Tipo	Restricciones	Descripción
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Identificador
usuario_id	INTEGER	NOT NULL, FK → usuarios(id), ON DELETE CASCADE	Dueño del caso
cliente_id	INTEGER	NOT NULL, FK → clientes(id), ON DELETE CASCADE	Cliente al que pertenece
asunto	TEXT	NOT NULL	Título del caso
descripcion	TEXT	—	Detalle (opcional)
prioridad	TEXT	NOT NULL, CHECK (Alta, Media, Baja)	Prioridad
estado	TEXT	NOT NULL, CHECK (Abierto, En proceso, Cerrado)	Estado
fecha_limite	TEXT	NOT NULL	Fecha límite (yyyy-MM-dd)
fecha_creacion	TEXT	NOT NULL	Fecha de creación (yyyy-MM-dd)

Se crean índices sobre `clientes.usuario_id`, `casos.usuario_id` y `casos.cliente_id` para acelerar las consultas.

6.5 Decisiones de diseño

- **Llaves foráneas activas.** SQLite no las activa por defecto, así que se activan en `onConfigure()` con `setForeignKeyConstraintsEnabled(true)`. Sin esto, `ON DELETE CASCADE` no funcionaría.
- **Eliminación en cascada.** Al eliminar un cliente se borran sus casos automáticamente, y la app lo advierte antes de confirmar.
- **Restricciones `CHECK`.** La base de datos solo acepta prioridades y estados válidos, aunque la app ya los limita con botones de opción. Es una doble validación.
- **Fechas en formato ISO (yyyy-MM-dd).** En este formato el orden alfabético coincide con el cronológico, lo que permite ordenar por fecha y detectar vencidos comparando texto (`fecha_limite < hoy`).

7. Lógica principal

7.1 Registro e inicio de sesión

1. **Registro:** se validan los campos y que el correo no exista (`existeCorreo`). La contraseña se convierte a un hash SHA-256 y se guarda con `registrarUsuario`. Después se vuelve al login con el correo ya escrito.
2. **Login:** se calcula el hash de la contraseña escrita y se busca un usuario con ese correo y ese hash (`login`). Si existe, se guarda su `id` y su nombre en `SharedPreferences` (`SessionManager`), así la sesión se mantiene aunque se cierre la app.
3. Todas las consultas posteriores usan ese `usuario_id`, así cada persona solo ve su información.

7.2 Ciclo de interacción

Cada formulario sigue el mismo ciclo de cuatro pasos:

1. **Acción:** el usuario toca un botón, por ejemplo “Guardar”.
2. **Escucha:** un `setOnClickListener` detecta el evento.
3. **Lógica:** se validan los datos. Si hay errores, se muestran debajo de cada campo (`TextInputLayout.setError`).
4. **Mutación:** se guarda en SQLite y se actualiza la interfaz con un mensaje (`Snackbar`) y la lista recargada, sin reiniciar la app. Cuando la pantalla se cierra al guardar (por ejemplo, el formulario de casos), le devuelve el mensaje a la pantalla anterior con la API `ActivityResult`, y esa lo muestra.

7.3 Lista de casos, búsqueda y filtros

- La lista se carga con `buscarCasos(usuarioId, texto, estado)`, que hace un **JOIN** entre `casos` y `clientes` para mostrar el nombre del cliente en cada fila.
- La búsqueda usa `LIKE '%texto%'` sobre el asunto, el nombre del cliente y la empresa. Se ejecuta con cada letra escrita (`TextWatcher`). Los caracteres `%` y `_` que escribe el usuario se escapan (`ESCAPE '\'`) para que se busquen como texto y no como comodines.
- La lista usa `ListAdapter` con `DiffUtil`: al filtrar, solo se animan las filas que cambian.
- Los *chips* filtran por estado (`AND estado = ?`).
- El orden es por prioridad (Alta → Media → Baja) y después por fecha límite más cercana.
- La lista se recarga en `onResume()`, así siempre refleja los cambios hechos en otras pantallas.

7.4 Resumen y casos vencidos

- `obtenerResumen` calcula en una sola consulta (`SUM(CASE WHEN ...)`) cuántos casos hay abiertos, en proceso, cerrados y vencidos.
- Un caso está **vencido** si su estado no es “Cerrado” y su fecha límite es anterior a hoy. Se marca en rojo en la lista y en el detalle.

7.5 Clientes

- `obtenerClientes` usa un `LEFT JOIN` con `COUNT` para mostrar cuántos casos tiene cada cliente. El `LEFT JOIN` incluye también a los clientes que aún no tienen casos.
- Crear y editar se hace en una hoja inferior (`BottomSheetDialog`). El botón “Guardar” valida antes de cerrarla, y desde la misma hoja se elimina el cliente.
- La lista de clientes tiene su propia búsqueda por nombre o empresa, que ignora mayúsculas y tildes.
- Para crear un caso se necesita al menos un cliente. Si no hay ninguno, la app lo indica y lleva a la pantalla de clientes.

8. Diseño de interfaz

La interfaz sigue un estilo **minimalista**: fondo blanco, tinta casi negra, grises neutros y un solo color de acento. Sin degradados, sin sombras decorativas y sin emojis. La jerarquía se marca con el tamaño y el peso del texto y con el espacio, no con colores.

- **Tema Material 3** (`Theme.Material3.Light.NoActionBar`) en `res/values/themes.xml`. Se reemplazan los tonos lavanda que Material 3 trae por defecto por superficies neutras, así diálogos, calendario y menús también se ven limpios.
- **Paleta** en `res/values/colors.xml`: acento azul `#1E3A8A`, tinta `#111111`, texto secundario `#6B7280`, líneas `#E5E7EB`. Todos los pares de texto y fondo cumplen contraste **WCAG AA** (4.5:1 o más). El borde de los campos de texto cumple 3:1.
- **Tipografía Inter** en tres pesos: 400 para el cuerpo, 500 para énfasis y 600 para títulos. La escala va de 12 a 32 sp.
- **Estilos reutilizables** en `res/values/styles.xml` (equivalentes a clases CSS): textos (`Text.ClienTrack.*`), botones, campos, chips, segmentos, barra superior y apariencias de texto para los componentes de Material.
- **Etiquetas tonales** para el estado del caso: fondo claro y texto oscuro del mismo color (Abierto azul, En proceso violeta, Cerrado gris, Vencido rojo). La prioridad se muestra con un punto de color y su texto, una sola vez por fila.
- **Listas como filas** separadas por líneas de 1 dp, en lugar de tarjetas apiladas.
- **Controles:** grupos segmentados para prioridad y estado, menú desplegable para el cliente, calendario de Material para la fecha límite, hoja inferior para el formulario de

clientes y botón de acción con texto (“Nuevo caso”) que se encoge al bajar por la lista.
- **Estados vacíos** con ícono, título y una línea de ayuda.
- **Accesibilidad:** áreas táctiles de 48 dp o más, etiquetas encima de cada campo (`labelFor`), descripciones para los botones de solo ícono y para los lectores de pantalla.
- **Íconos de trazo** dibujados como vectores (`res/drawable/ic_*.xml`), basados en Feather Icons (licencia MIT).
- **Diseño adaptable:** se usan `match_parent`, pesos (`layout_weight`) y vistas con desplazamiento, así las pantallas se ajustan a distintos tamaños de celular. Los formularios suben con el teclado (`adjustResize`).

9. Pruebas

Tipo	Dónde	Qué prueba
Unitarias (JUnit)	`app/src/test`	Validaciones (correo, teléfono, nombre, asunto, contraseña), hash SHA-256, formatos de fecha, conversión de fechas del calendario, iniciales, búsqueda sin tildes, escape de comodines y regla de caso vencido. 19 pruebas.
Interfaz (Espresso)	`app/src/androidTest`	Bienvenida; errores del login; contraseñas distintas en el registro; registro e inicio de sesión; estado vacío; aviso de “primero agrega un cliente”; validaciones del formulario de casos; crear, cerrar y eliminar un caso. 7 pruebas.

El archivo **`probar.bat`** (Windows) o **`probar.sh`** (macOS/Linux) revisa el entorno (Java 17 o superior, Android SDK, ruta sin tildes), compila el APK, ejecuta las pruebas unitarias y lint, y, si hay un emulador abierto, las pruebas de interfaz. Al final muestra un resumen y deja el APK en `salida/ClienTrack-debug.apk`.